Project Sentinel — Implementation Deep-Dive

What was built, why these approaches, the tradeoffs, and the concepts in depth

June 2026

Contents

1. Orientation
2. The architecture in one page
3. Engineering philosophy (the cross-cutting approaches)
   - 3.1 Test-Driven Development (RED → GREEN → REFACTOR)
   - 3.2 Seams and dependency injection
   - 3.3 Typed contracts, validated at every hop
   - 3.4 Offline-by-default, infrastructure opt-in
   - 3.5 Purity and determinism for the decision layer
4. The build, component by component
   - 4.1 The domain vocabulary — `enums.py`, `ids.py`, `contracts/`
   - 4.2 The escalation engine — `escalation.py` (*the product-defining code*)
   - 4.3 Confidence: blend + calibration — `confidence.py`
   - 4.4 Edge-side redaction (C2) — `redaction.py`
   - 4.5 Connectors and correlation — `connectors/`, `correlator.py`
   - 4.6 The orchestrator and the LangGraph runtime — `orchestrator/`, `graph/`
   - 4.7 The LLM seam and cost governance — `agents/llm.py`, `agents/meter.py`, `config.py`
   - 4.8 Multi-tenant isolation: Postgres Row-Level Security — `data/` (*the headline security proof*)
   - 4.9 Event-sourced incidents — `incidents/`, `data/repository.py`
   - 4.10 Calibration and Expected Calibration Error — `eval/calibration.py`
   - 4.11 Tamper-evident audit log: a hash-chain — `audit.py`, `data/audit.py`
   - 4.12 L2 LLM-as-judge — `eval/judge.py`
   - 4.13 Durable inbox/outbox — `data/inbox_outbox.py`
   - 4.14 The golden incident library + L1 evals — `eval/golden.py`, `golden/`, with `escalation.py`
5. The tradeoffs ledger
6. What is deliberately *not* built (and why)
7. How it is verified
8. Concept glossary

1. Orientation

Project Sentinel is an agentic on-call SRE delivered as a multi-tenant SaaS: it watches a small team's production stack, and when something breaks it diagnoses the root cause, drafts the fix, and either applies it (if explicitly trusted to) or pages a human with the entire investigation already done. The hard part is not

"can an LLM diagnose an incident" — it is doing that **against paying strangers' production infrastructure, across many tenants, without ever being the cause of an outage and without ever seeing data we shouldn't.**

Three inviolable constraints shape every decision in the codebase:

- **C1 — Never be the cause of an outage.** Read-only by default; every write is opt-in, allow-listed, reversible, and approval-gated.
- **C2 — Minimize what we can see.** Secrets and PII are redacted *at the ingestion edge, before anything persists.*
- **C3 — Onboard in minutes.** Time-to-first-value is the product's survival metric.

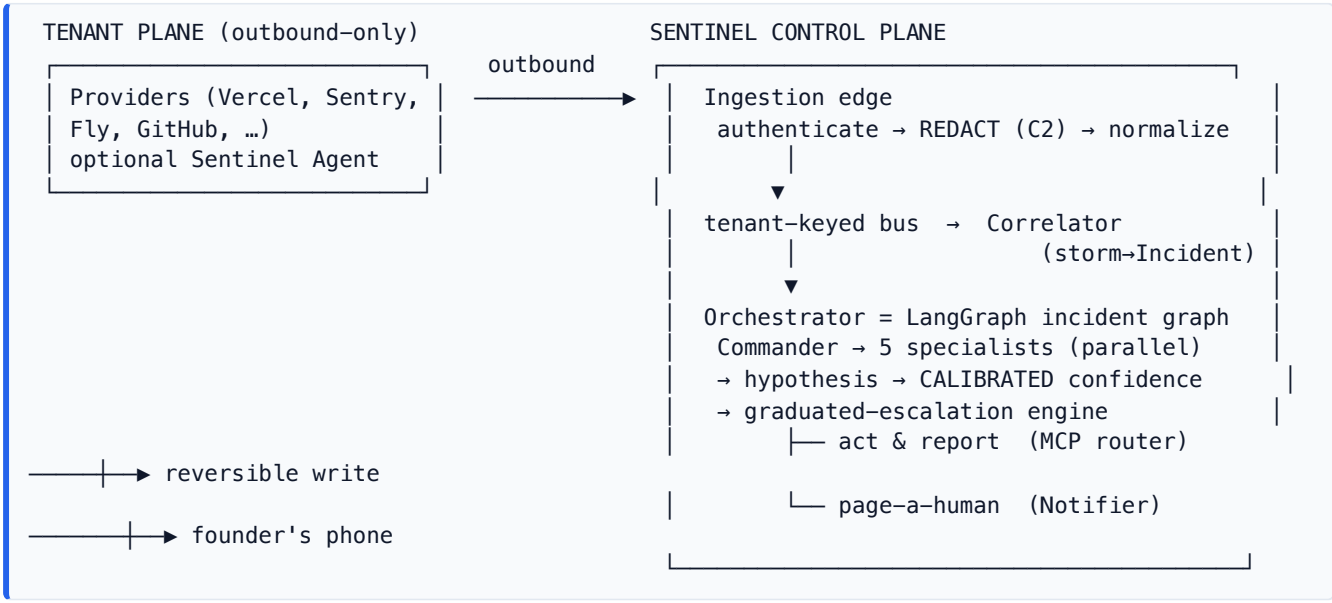
This document explains the **implementation** that backs that design: a fully typed, offline-by-default Python core with the security- and correctness-critical guarantees *proven by tests*, not merely asserted. For each component it answers four questions: **what** it is, **why** this approach, **what concepts** underlie it (from first principles), and **what tradeoffs** the choice carries.

The build currently stands at **297 tests, all green**, `mypy --strict` clean across 83 files, `ruff` clean. The two real external dependencies — PostgreSQL and the Anthropic API — are both opt-in and gated, so the entire suite runs with neither present (the dependent tests skip cleanly).

---

2. The architecture in one page

Two planes separated by a single trust boundary, with one canonical event carrying an incident from a provider webhook to an outcome:



The **graduated-autonomy ladder** is the product itself — trust is climbed deliberately, per action type:

Tier	Name	Behavior
T0	Observe	Read-only; diagnose and page only. *(default)*
T1	Suggest	Drafts a fix; never executes.
T2	Approve-to-act	Executes an allow-listed, reversible action after one-tap approval.
T3	Autonomous	Auto-executes allow-listed, reversible, **calibrated-HIGH-confidence** actions, then reports.

The single most important behavior in the system: the **only** path to an autonomous write is `reversible AND calibrated-HIGH AND tenant-granted-T3`. Everything else pages a human — but with the root cause, evidence chain, competing hypotheses, and a drafted fix already attached, so approval takes ~15 seconds instead of a cold 3 a.m. investigation. Irreversible actions and low-confidence diagnoses *always* page; no granted tier overrides those hard stops.

---

3. Engineering philosophy (the cross-cutting approaches)

Five disciplines recur in every component. They are worth stating up front because they explain *why the code looks the way it does.*

3.1 Test-Driven Development (RED → GREEN → REFACTOR)

Every behavior was written test-first: write a failing test, **watch it fail for the right reason**, write the minimum code to pass, then refactor. This is not ceremony — for a system whose selling point is "safe enough to touch production," a test that was written *after* the code merely confirms the code does what the code does. A test written *first*, watched failing, proves it actually constrains behavior.

The most valuable expression of this is the **RED→GREEN isolation proof** (§4.8): the cross-tenant security test is run once *without* Row-Level Security and watched to **fail** — proving it detects a breach — then *with* RLS and watched to pass. A test that has never failed is not evidence.

Tradeoff: test-first is slower per line up front and demands that designs be testable. The payoff is that "testable" and "well-factored" turn out to be the same property — code that is hard to test is hard to use.

3.2 Seams and dependency injection

Every expensive or non-deterministic dependency (an LLM call, a database, a clock, a notifier) is injected behind a narrow interface rather than reached for directly. The LLM, for instance, is a single function type — `Completer = (label, system, user) -> str` — so all the JSON-parsing and contract-building logic is unit-tested offline against a fake completer for **free**, while the thin real-network adapter is the only thing a live run exercises. The judge (§4.11) and the calibrator follow the same shape.

Why: it makes the bulk of the logic *deterministic and free to test*, and it isolates the parts that cost money or require infrastructure to a thin, easily-audited edge. **Tradeoff:** more interfaces, a little indirection. For a system where most logic must be exercised thousands of times in CI without spending money, this is decisively worth it.

3.3 Typed contracts, validated at every hop

Agents communicate only through Pydantic v2 models (`SpecialistFinding`, `Hypothesis`, `ConfidenceAssessment`, `RemediationPlan`, `EscalationDecision`), never free text, and the whole codebase passes `mypy --strict`. A malformed agent output is a *validation error at the boundary*, not a mysterious failure three functions later. This is what makes the system debuggable, evaluable, persistable, and brokerable.

Tradeoff: strict typing and `extra="forbid"` schemas are more rigid; they reject "convenient" loose dicts. That rigidity is the point — it converts a class of runtime bugs into impossible states.

3.4 Offline-by-default, infrastructure opt-in

The core depends only on Pydantic. Kafka is modeled by an in-memory tenant-keyed bus; the database, the LLM, and LangGraph are optional extras. Tests that need Postgres or a key **skip cleanly** when those are absent. This keeps the inner development loop fast and free, and means the project is runnable by anyone with just Python.

Tradeoff: in-memory fakes are not the real thing — they can hide concurrency or durability bugs the real backends would surface. The mitigation is that the *seams* are real (the same interfaces a production backend implements), and the genuinely un-fakeable guarantees (RLS isolation, durable hash-chains, exactly-once) are tested against **real Postgres**, not a fake.

3.5 Purity and determinism for the decision layer

The diagnosis is probabilistic; the **decision to touch production is not.** The escalation engine is a pure, total function of three inputs. The confidence calibration map and the projection fold are pure. Purity makes the safety-critical layer exhaustively testable, trivially auditable, and immune to LLM nondeterminism — you can enumerate the entire decision space and assert C1 holds in every cell.

---

4. The build, component by component

This section walks the implemented modules. For each: **what / why / concepts / tradeoffs.**

4.1 The domain vocabulary — `enums.py`, `ids.py`, `contracts/`

What. The canonical strings (`Severity`, `EventKind`, `AutonomyTier`, `ConfidenceBand`, `IncidentState`, `ActionType`) as `StrEnum`s, k-sortable ULID identifiers, and the Pydantic inter-agent contracts. A single `band_for(confidence)` applies the canonical cut `LOW < 0.5 ≤ MEDIUM < 0.8 ≤ HIGH`.

Why. One source of truth referenced verbatim by the escalation engine, persistence, and connectors prevents the classic drift where "HIGH" means one thing in the policy and another in the database. `StrEnum` makes the values both type-safe in Python and clean as JSON/SQL literals (and lets YAML keys coerce directly to enum members in the eval layer).

Concept — k-sortable IDs. A ULID is 128 bits like a UUID but its leading bits are a timestamp, so IDs sort in creation order. Stored as `uuid`, rendered with a prefix (`inc_…`, `evt_…`). This gives human-readable, globally-unique, time-ordered keys without a central sequence.

4.2 The escalation engine — `escalation.py` (*the product-defining code*)

What. `decide_escalation(PolicyInputs) -> EscalationDecision`: a pure function mapping `(calibrated band × granted tier × reversibility)` to exactly one of five outcomes — `act_and_report`, `page_one_tap`, `page_with_drafted_fix`, `page_irreversible`, `page_low_confidence`.

Why a pure function. This is the §00.6 matrix — the thing that decides whether to touch prod. Making it a pure, total function (no I/O, no LLM, no clock) means it is **exhaustively unit-testable**: the test suite enumerates the entire Cartesian product of `(band, tier, reversible)` and asserts the C1 invariant in every cell — *the only autonomous-write path is* `reversible AND HIGH AND T3`.

Concept — ordering encodes safety priority. The two hard stops (irreversible, LOW confidence) are evaluated **before** any tier check, so no amount of granted autonomy can override them. The order of `if` statements *is* the safety policy.

Tradeoff. A matrix is rigid — it cannot express "usually safe" nuance. That rigidity is exactly what you want for a production-write gate: every input yields one predictable, auditable outcome.

4.3 Confidence: blend + calibration — `confidence.py`

What. `blend_raw_likelihood(self_assessment, corroboration_count, n_competing)` fuses three signals into a raw likelihood; a versioned `Calibrator` maps it through `sigmoid(logit(raw) / T)`; `band_for` cuts the calibrated number into a band.

Why blend this way. Corroboration across independent specialists is the most predictive signal, so the formula closes the gap to 1.0 with diminishing returns per corroboration; competing hypotheses *cap* confidence multiplicatively (two near-tied root causes must not yield HIGH no matter how confident the model sounds). The formula is deliberately simple, documented, and **monotonic** so its behavior is obvious.

Concept — why raw model confidence is a lie. A language model that says "0.9" is right far less than 90% of the time; models are *systematically over-confident*, and differently so per model. Feeding raw 0.9 into a policy that may then touch prod is the "confidence is vibes" failure. The fix — calibration — is detailed in §4.10.

Tradeoff. The blend is a hand-designed heuristic, not a learned function. It is interpretable and needs no training data, at the cost of not capturing subtle interactions a learned scorer might. The calibration map (fit from data) is where empirical correction enters.

4.4 Edge-side redaction (C2) — `redaction.py`

What. Layered detectors (known-key patterns plus a Shannon-entropy heuristic for high-entropy tokens) strip secrets/PII and emit `redaction_meta` recording *what kind* was removed and how many — never the value.

Why at the edge. Redaction happens *before* anything is written to the bus, the database, or sent to an LLM. The library inherits its secret-free property from the ingestion boundary, not a later scrubbing pass that could be forgotten. The demo proves it end-to-end: the real Claude run only ever sees `{'pg_password': 1, 'pii.phone': 1}`-style metadata, never the secret.

Concept — precision/recall and deliberate over-redaction. A detector can err two ways: miss a secret (false negative — catastrophic) or scrub a benign token (false positive — annoying). Sentinel tunes toward **over-redaction**: a needlessly-masked token is a cosmetic loss; a leaked credential is a breach. The entropy detector exists precisely to catch secrets that match no known pattern.

4.5 Connectors and correlation — `connectors/`, `correlator.py`

What. A `Connector` Protocol with capability-as-data, Sentry/Vercel adapters, and webhook idempotency helpers; the correlator collapses a storm of events into one `Incident`, including a cross-provider join on `commit_sha` (a Vercel deploy webhook and a Sentry error spike tagged with the same release become one incident).

Why a Protocol. New providers are the moat, and each differs (Vercel has deploy webhooks; Fly has none and must be polled). A structural `Protocol` lets every provider be a small adapter behind one interface, with capabilities expressed as *data* the orchestrator can read, rather than branching logic scattered through the core.

Concept — correlation vs. deduplication. Dedup removes *identical* re-deliveries (§4.12). Correlation is semantic: many *distinct* signals about one underlying failure are grouped by time window, service topology, and shared identifiers so the agent investigates one incident, not forty alerts.

4.6 The orchestrator and the LangGraph runtime — `orchestrator/`, `graph/`

What. A runtime-agnostic incident-graph driver, plus a real **LangGraph** realization: a checkpointed `StateGraph` whose nodes are intake → parallel specialists → synthesize → score → decide → (await-approval) → act, with a genuine human-in-the-loop `interrupt()` at the T2 approval gate.

Why LangGraph + interrupt. A T2 incident must *pause* for human approval and *resume* — possibly minutes later, possibly after a process restart. LangGraph's checkpointer persists graph state at each step; `interrupt()` suspends the run and `Command(resume=…)` continues it from the saved checkpoint. In production the checkpointer is swapped for the Postgres one with a single argument; in tests an in-memory saver exercises the identical control flow.

Concept — durable execution and gap-free sequencing. The incident's `event_log` is sequenced with a contiguous `seq`. The graph is designed so only *sequential* nodes emit sequence-advancing events while the *parallel* specialists return findings only — so the `seq` stays gap-free **across the suspend/resume boundary**, preserving the event-sourced replay invariant (§4.9). The interrupt re-runs its node from the top on resume, so the `interrupt()` call must be the node's first statement with no side effects before it — a subtle correctness requirement that drove the node design.

Tradeoff. LangGraph adds a dependency and a learning curve versus a hand-rolled state machine. The payoff is battle-tested durable checkpointing and HITL primitives that would be error-prone to reimplement, plus a clean production swap to Postgres-backed durability.

4.7 The LLM seam and cost governance — `agents/llm.py`, `agents/meter.py`, `config.py`

What. `LLMSpecialist` / `LLMCommander` depend only on the `Completer` seam; `anthropic_completer` is the metered real adapter; `TokenMeter` records tokens and **dollar cost per call** (with cache-read/write pricing); `Settings.llm_ready` gates everything (off unless a key is present *and* `SENTINEL_USE_LLM=1`).

Why metered and gated. This is a cost-conscious project on someone else's budget: a runaway loop of LLM calls is both a financial and a safety concern (per-tenant cost ceilings are an ADR-level control). Every call's cost is surfaced; a real end-to-end incident run costs ≈ **$0.003** and says so. The seam means the parsing/tolerance logic (extracting JSON from prose, clamping out-of-range confidences, rejecting booleans-as-numbers) is all tested offline.

Concept — cheap model by default, strong model where it matters. The diagnosing specialists default to the cheapest model (Haiku); the *judge* (§4.11) deliberately uses a stronger, different model — it must not grade its own homework.

4.8 Multi-tenant isolation: Postgres Row-Level Security — `data/` (*the headline security proof*)

What. A real PostgreSQL schema with the design's tables, two database roles, and a `tenant_isolation` Row-Level-Security policy on every tenant-scoped table, keyed on a per-request session variable. Plus a `tenant_session` context manager that sets that variable transactionally, and a five-test isolation suite proving tenant A cannot read tenant B.

Why RLS instead of "just filter by tenant_id in the app." An application-layer rule (`WHERE tenant_id = …`) is *one* check, in *one* place, that a single forgotten `.filter()`, a raw SQL string, a new endpoint, an ORM lazy-load, or a SQL-injection foothold can defeat. RLS moves the check **below** all application code, into the database engine, where *every* statement on *every* connection is constrained regardless of how the SQL was built. The two layers are independent — a bug in one is not sufficient to cross tenants. This is **defense in depth**, and it is *why* a shared-database ("pool") multi-tenancy model is safe at all.

Concepts, from first principles:

- **Multi-tenancy models.** *Pool* (all tenants share tables, isolated by a column + RLS), *bridge* (shared DB, separate schemas), *silo* (a database per tenant). Pool is cheapest and onboards instantly (C3) but is only safe with a hard isolation mechanism — hence RLS.
- **The session GUC.** Isolation keys off a custom setting, `app.tenant_id`, set per transaction with `SET LOCAL` (via `set_config(..., true)`). `SET LOCAL` ties the value to the current transaction and auto-resets on commit/rollback — essential with a connection pool, because a leaked setting would itself be a cross-tenant leak. `set_config` is parameterized, so a hostile `tenant_id` is always a *value*, never SQL.
- **`USING` vs `WITH CHECK`.** The policy has both clauses: `USING` governs which rows are *visible* (SELECT/UPDATE/DELETE); `WITH CHECK` governs which rows may be *written* (INSERT/UPDATE). Without `WITH CHECK`, a tenant could *insert* a row stamped with someone else's `tenant_id` — a write-side cross-tenant attack. Both clauses compare `tenant_id = current_setting('app.tenant_id', true)::uuid`.
- **Fail-closed.** `current_setting(..., true)` returns `NULL` when the GUC is unset (rather than erroring), and `NULL` matches *nothing* — so a connection that forgot to set its tenant scope sees **zero rows**, never everything. Fail-closed, not fail-open.
- **`NOBYPASSRLS` and `FORCE ROW LEVEL SECURITY`.** The single most important line in the data model is `NOBYPASSRLS` on the role the application actually connects as — a superuser or `BYPASSRLS` role would render every policy cosmetic. `FORCE RLS` additionally binds the *table owner* (who is otherwise exempt), closing the migration-job-as-owner footgun.
- **The forged-filter case.** The decisive test: while scoped to tenant A, run an explicit `SELECT … WHERE tenant_id = B` (an attacker who *knows* B's id). It returns nothing, because Postgres **ANDs** the policy predicate onto the query's own — visibility can only ever *narrow*. You cannot widen it by supplying a filter.

Why split grants from policies. Grants decide *which verbs* a role may use; RLS decides *which rows*. Keeping them in separate SQL files is what lets the isolation test run *with table access but no row filtering* — the RED step. It also makes append-only tables real: the app role is granted no `DELETE` / `UPDATE` on the

event store or audit log, so history cannot be rewritten even by a bug.

The RED→GREEN methodology. The suite is run twice. With RLS disabled, `test_a_cannot_select_b`, `test_forged_filter_is_powerless`, `test_cannot_insert_for_another_tenant`, and `test_missing_guc_is_fail_closed` all **fail** — proving they genuinely detect leakage. With RLS enabled, all pass. The one test that passes in both states (`test_app_role_has_no_bypass`, which asserts the runtime role lacks `BYPASSRLS`) is the guard that keeps the whole guarantee honest.

Tradeoffs. RLS adds a small per-transaction cost (setting the GUC) and forces transaction-scoped sessions everywhere. `LIST(tenant_id)` partitioning (designed, not yet built) is graduation-friendly but Postgres planning degrades past a few thousand partitions. These are accepted: the isolation guarantee is worth a GUC set per transaction, and "whales graduate to silo before partition count bites" is the scaling answer.

4.9 Event-sourced incidents — `incidents/`, `data/repository.py`

What. The `incident_events` table is an **append-only log**; the `incidents` row is a **projection** rebuilt by folding events in `seq` order. `append_incident_event` writes the next event and updates the projection in one transaction; `rebuild_incident` reconstructs the projection from scratch by re-folding the log.

Why event sourcing. It buys three things the design needs: an **audit trail** (every state change is a durable, ordered fact — who did what, when), **restart-survival** (the projection is reconstructable, so a crash loses nothing), and **replayable evals** (the golden-incident library, §4.10, replays event streams). The fold is a *pure* function `(projection, event) → projection`, so the same log always yields the same projection — the foundation of replay.

Concept — projection as a cache. The projection is just a materialized, fast-to-read view of the log; the log is the source of truth. `rebuild_incident` deliberately overwrites the projection in place via `UPDATE` rather than `DELETE`+re-insert, because the app role is granted no `DELETE` on the projection — respecting that least-privilege grant is the point, and it proves the rebuild needs no elevated role.

Tradeoff. Event sourcing is more moving parts than a CRUD row you mutate directly: you maintain a fold and accept eventual-vs-immediate projection semantics. For an incident system where *the history is itself a product* (the audit story) and replay drives evals, the log-as-truth model pays for itself.

4.10 Calibration and Expected Calibration Error — `eval/calibration.py`

What. A from-scratch, pure-Python measurement of how *honest* the confidence numbers are: reliability binning, **Expected Calibration Error (ECE)**, the negative-log-likelihood objective, and a **temperature fit** by golden-section search. The headline test fits a temperature on an over-confident model and shows its ECE collapse toward zero.

Concepts, from first principles:

- **Calibration.** A predictor is *calibrated* if, among all the times it says "0.8," it is right about 80% of the time. Calibration is what makes the band labels *mean* something: "HIGH" must correspond to genuinely high historical accuracy, or the C1 guarantee for autonomous action is built on sand.
- **Reliability diagram.** Bin predictions by stated confidence; in each bin plot *mean confidence* (x) against *observed accuracy* (y). A perfectly calibrated model lies on the diagonal. Over-confidence shows as points *below* the diagonal (it claims more than it delivers).
- **ECE.** A single number summarizing the diagram: the accuracy-weighted average gap between confidence and accuracy across bins, `Σ (|bin| / N) · |conf_bin − acc_bin|`. It is the headline metric a CI gate can cap.

- **Temperature scaling.** The simplest, most robust calibration map: divide the *logit* of the confidence by a scalar `T` before re-applying the sigmoid — `sigmoid(logit(p) / T)`. `T > 1` *softens* over-confident scores toward 0.5 (the usual case); `T = 1` is the identity; `T < 1` sharpens. It is monotonic (it never reorders predictions, so it cannot make accuracy worse) and has exactly one parameter, so it fits from very little data.
- **Why NLL and golden-section.** `T` is fit by minimizing **negative log-likelihood** — the standard proper scoring rule that rewards being confident *and right* and punishes being confident *and wrong*. NLL as a function of `T` is unimodal on a bounded interval, so a **golden-section search** (a derivative-free 1-D minimizer that shrinks the bracket by the golden ratio each step) finds the optimum deterministically in a handful of evaluations — no gradients, no libraries.

Why pure-Python (no numpy/scipy). The reference design sketches this in numpy/scipy, but the entire calibration core is implemented in the standard library's `math` only, matching `confidence.py`'s stdlib-only stance. **Why:** it keeps the dependency surface tiny, it is more clearly *understood* (you can read every line of the ECE sum and the minimizer), and for a portfolio project "I implemented ECE and an NLL-minimizing temperature fit from scratch" demonstrates the concept far better than "I called `scipy.optimize`." **Tradeoff:** numpy would be faster on huge arrays and scipy's minimizer is more general. At eval-set scale neither matters, and determinism (no BLAS nondeterminism) is a bonus.

4.11 Tamper-evident audit log: a hash-chain — `audit.py`, `data/audit.py`

What. A per-tenant cryptographic hash-chain over the `audit_log` table. Each entry's `entry_hash = SHA-256(seq ‖ prev_hash ‖ canonical(row))`, where `‖` is length-prefixed concatenation and `prev_hash` links to the previous entry (genesis = 32 zero bytes). `verify_chain` walks the log, recomputes every hash, and reports the first break. The pure mechanism is offline; a durable variant appends to and verifies over Postgres.

Concepts, from first principles:

- **Hash chain.** Because each entry's hash includes the *previous* entry's hash, the entries form a chain like a linked list secured by cryptography. Altering or deleting any historical entry changes its hash, which breaks the `prev_hash` link of *every* later entry. To hide a single edit you would have to recompute every subsequent row — and the application role *cannot*, because it has no `UPDATE` / `DELETE` grant on the table (the append-only guarantee from §4.8). So the chain is tamper-**evident** at the crypto layer and tamper-**resistant** at the privilege layer, simultaneously.
- **Canonical serialization.** The hash is taken over a *deterministic* encoding of the semantic fields — sorted keys, fixed separators, UTC timestamps — so anyone holding the row can recompute the exact bytes. This is what makes the chain **independently verifiable** and **tenant-exportable**: a security-conscious customer can export their chain and confirm the head hash with any SHA-256 implementation. "Show me every time you touched my prod, and prove the list wasn't edited."
- **Length-prefixed framing.** Concatenating `seq ‖ prev_hash ‖ row` naively is ambiguous (different field splits could produce identical byte streams). Prefixing each field with its 8-byte length removes that ambiguity so two different logical entries can never hash-collide by framing accident.

Why a hash-chain rather than per-row signatures or a Merkle tree. Signatures prove *authorship* but not *ordering or completeness* — you could drop a signed row undetected. A hash-chain proves the *sequence* is intact and complete, which is exactly the audit property ("nothing was inserted, deleted, or reordered"). A Merkle tree gives efficient *inclusion proofs* for huge logs but is overkill here, where full linear verification is

cheap and the export story wants simple reproducibility. The head hash can later be periodically *anchored* to an external append-only store so even a control-plane operator with DB write access cannot rewrite history without externally-provable divergence.

Tradeoff. Verification is O(n) in the log length (fine for per-tenant audit volumes; a Merkle tree would be the move if logs grew to millions of entries and partial proofs were needed). Hashing is not encryption — the log is integrity-protected, not confidential; confidentiality is handled separately by storing only redacted `detail` (the log records *authority spent*, never *data seen*).

4.12 L2 LLM-as-judge — `eval/judge.py`

What. A second evaluation layer that scores an end-to-end diagnosis against golden ground truth on a **weighted rubric** (root-cause correctness 0.40, evidence soundness 0.25, remediation actionability 0.20, calibration honesty 0.10, no-hallucinated-evidence 0.05), returning a typed `RubricScore`. Hallucinated evidence is a **hard veto** — a 0 there fails the diagnosis outright regardless of the weighted total.

Why a judge at all. Root-cause quality is not exact-match-able: two correct diagnoses can be worded differently and a wrong one can be fluent. Deterministic assertions (L1) can check *structure* — did the right specialist fire, is the evidence redacted, is the escalation verdict right — but not open-ended *quality*. An LLM judge scores quality against a rubric.

Concepts:

- **Judge ≠ diagnoser.** The judge uses a deliberately *stronger and different* model than the agents it grades — it must not grade its own homework. It sees only redacted contracts (C2).
- **The hallucination veto encodes C1.** Inventing a signal to justify a write is precisely the failure that could cause an outage, so it cannot be averaged away by strong scores elsewhere; it is a categorical fail.
- **Who watches the judge.** A drifting judge is a silent eval failure, so the design pins a small human-graded meta-eval set and requires the judge's agreement with human labels (Cohen's κ) to stay above a floor before it may grade anything. (Designed; the rubric and scoring are implemented and unit-tested with a fake judge.)

Why the seam. As with all LLM calls, `judge_diagnosis` depends on a `Judge` seam, so the rubric math, the weighting, and the veto are unit-tested offline with a fake judge returning canned JSON; the real Anthropic adapter is lazily imported so the module loads without the SDK present. **Tradeoff:** an LLM judge is itself probabilistic and costs money — hence it is opt-in, gated, and guarded by its own meta-eval, while L1 (free, deterministic) does the bulk of the gating.

4.13 Durable inbox/outbox — `data/inbox_outbox.py`

What. Two tenant-scoped, RLS-governed tables: `webhook_inbox` (unique on `(integration_id, idempotency_key)`) dedups provider re-deliveries; `event_outbox` holds normalized events for a relay to drain. `accept_webhook` records the delivery and enqueues events *in one transaction*; a duplicate delivery hits the unique constraint, does nothing, and is acknowledged.

Concepts, from first principles:

- **At-least-once delivery + idempotent consumer.** Providers retry on any non-2xx or timeout, so the same webhook can arrive many times. The standard correctness pattern for "messages arrive at least once" is to make processing **idempotent**: a deterministic idempotency key (prefer the provider's own delivery id; fall back to a content hash of the *semantically identifying* fields, excluding volatile

timestamps) plus a unique constraint turns "process every POST" into "process each logical event exactly once."

- **The dual-write problem.** If `normalize()` wrote to the database *and* published to the bus as two separate operations, a crash between them would either lose the event or double-publish it — you cannot atomically write to two systems. The **transactional outbox** solves this: the event is written to an outbox *table* in the *same database transaction* as the inbox insert, and a separate relay later publishes from the outbox. The publish is then **exactly-once relative to the inbox decision** even across a crash, because the decision and the to-be-published event commit atomically together.
- **Replay falls out for free.** Because the inbox key is deterministic and the outbox is durable, re-feeding a stored raw payload is safe (the inbox dedups true duplicates), which is the same primitive the golden-incident replay uses.

Tradeoff. The outbox adds a table and a relay step versus publishing directly — more machinery, slightly more latency. It is the price of not losing or duplicating an event when a process dies at the wrong moment, which for an incident pipeline (duplicate incidents, duplicate pages, or a double *write* — a C1 violation) is non-negotiable.

4.14 The golden incident library + L1 evals — `eval/golden.py`, `golden/`, with `escalation.py`

What. A versioned corpus of YAML incident cases, **stratified by stack archetype** (edge/serverless+DB, container PaaS+Postgres, JAMstack, error-tracking-led, and a deliberately thin uptime-led stratum), loaded into typed `GoldenCase`s. Deterministic L1 tests assert per case: the smoking-gun specialist fires with a valid contract, every evidence reference is redacted, and the escalation verdict matches expectation for *every* grant tier.

Concepts:

- **Eval-driven development.** For an LLM system, traditional tests are necessary but insufficient — you also need a graded corpus that measures *diagnostic quality* and *catches regressions* as prompts and models change. The golden library is that corpus; L1 is its fast, deterministic, free tier.
- **Stratification.** A Vercel+Supabase app fails differently from a Fly+Postgres one (deploy-webhook-driven vs. poll-only). An eval set drawn from one stack silently over-fits the agents to that stack's signal shape, so the corpus enforces minimum coverage per archetype. The uptime-led stratum is intentionally thin-signal — it stress-tests the calibration logic exactly where a miscalibrated HIGH is most dangerous.
- **One case tests the whole matrix.** A case's `expected_outcome_by_grant` block asserts the escalation verdict for T0–T3 in a single replay, so one golden case exercises the entire safety matrix.

Why YAML + Pydantic. Cases are data, authored and reviewed like data; Pydantic validates them on load (a malformed case is a load-time error), and a `model_validator` stamps the case id onto each embedded finding so the YAML stays DRY. **Tradeoff:** embedding findings in the case (rather than re-running live specialists) makes L1 deterministic and free but means L1 tests the *pipeline and policy* against curated outputs; the *live* model quality is what L2 and a real run measure.

---

5. The tradeoffs ledger

A consolidated view of the major decisions:

Decision	Chosen	Alternative	Why chosen	Cost accepted
Tenant isolation	Postgres RLS (pool model)	App-layer `WHERE tenant_id`	Engine-enforced, defeats forgotten filters & injection	GUC per transaction; transaction-scoped sessions
Persistence shape	Event-sourced log + projection	Mutable CRUD rows	Audit trail, replay, restart-survival	More moving parts; fold to maintain
Calibration impl	Pure-Python ECE + golden-section	numpy/scipy	Tiny deps, fully understood, deterministic	Slower at huge scale (irrelevant here)
Audit integrity	SHA-256 hash-chain	Per-row signatures / Merkle tree	Proves order & completeness; simple export	O(n) verify (fine at audit volumes)
Exactly-once ingest	Transactional inbox/outbox	Direct publish after write	Survives a crash mid-flight	Extra table + relay step
LLM access	Injected `Completer` / `Judge` seam	Direct SDK calls in logic	Offline, free, deterministic unit tests	More interfaces/indirection
Decision layer	Pure total function (matrix)	LLM-decided escalation	Exhaustively testable; immune to nondeterminism	Rigid; no "fuzzy" nuance
Default posture	Offline; infra opt-in & gated	Require Postgres/Kafka to run	Fast free loop; runs anywhere	Fakes can hide backend-specific bugs
Confidence	Calibrated, never raw	Trust model's stated number	"HIGH" means real accuracy (C1)	Must fit & version a calibration map

---

6. What is deliberately *not* built (and why)

The implemented core proves every guarantee that can be demonstrated in a single-machine, offline-or-local environment. What remains is genuine infrastructure, deferred on purpose:

- **Kafka** — modeled by an in-memory tenant-keyed bus with the same partition-by-tenant semantics; the real broker is a deployment concern, not a logic one.
- **Redis** (incident locks, cost budgets, rate limits), **Qdrant** (per-tenant vector memory), **Vault** (secret management) — each needs a running service; their *interfaces* and *roles in the design* are specified.
- **Langfuse / production trace sampling (L3)** — needs a live production stream.
- **CI/CD eval-gated pipeline** — needs a CI host (deferred along with the GitHub integration).
- **Stripe billing** — the idempotent metering and per-tenant cost circuit-breaker *logic* are built; the payment integration is external.

The discipline here is honesty: these are modeled as injected interfaces with in-memory fakes, never faked-and-claimed-as-real. The genuinely un-fakeable properties (cross-tenant isolation, durable tamper-evidence, exactly-once) are tested against **real Postgres**.

---

7. How it is verified

- **297 tests**, `ruff` clean, `mypy --strict` clean (83 files), running in ~1.4 s.
- The **escalation matrix** is tested *exhaustively* over its entire input space, plus per golden case.
- The **isolation proof** is the RED→GREEN contrast: the same assertions fail without RLS and pass with it.
- The **calibration proof** constructs a known-over-confident dataset, fits `T`, and asserts ECE collapses below 0.02.
- The **hash-chain proof** tampers with a row and asserts `verify()` pinpoints the break; and asserts the app role *cannot* perform the tampering write at all.
- The **exactly-once proof** delivers the same webhook twice and asserts the bus sees the event once.
- Postgres-backed tests **skip** cleanly when no database is present, so the suite is green on any machine; a `SENTINEL_TEST_RLS=0` switch re-runs the isolation suite in its pre-RLS (failing) state on demand.

---

8. Concept glossary

- **C1 / C2 / C3** — the three product constraints: never cause an outage; minimize what we can see; onboard in minutes.
- **Graduated autonomy (T0–T3)** — per-action-type trust tiers; the only autonomous-write path is reversible + HIGH + T3.
- **Calibrated confidence** — a confidence number corrected so its value equals real historical accuracy.
- **ECE (Expected Calibration Error)** — accuracy-weighted mean gap between stated confidence and observed accuracy across bins.
- **Temperature scaling** — one-parameter calibration map `sigmoid(logit(p)/T)`; `T>1` softens over-confidence.
- **Logit** — the inverse-sigmoid of a probability, `log(p/(1−p))`; the natural space to rescale confidences.
- **NLL (negative log-likelihood)** — proper scoring rule minimized when fitting the calibration map.
- **Row-Level Security (RLS)** — Postgres feature where the engine appends a row-filter predicate to every statement.
- **GUC** — a Postgres "grand unified configuration" setting; here `app.tenant_id`, set per transaction.
- **`USING` vs `WITH CHECK`** — RLS clauses governing row *visibility* vs. row *writeability*.
- **Fail-closed** — on missing/ambiguous scope, deny (see nothing) rather than allow (see everything).
- **`NOBYPASSRLS` / `FORCE RLS`** — the role can't skip policies; even the table owner is bound.
- **Defense in depth** — independent layers such that a bug in one does not breach the system.
- **Event sourcing** — store an append-only log of events; derive state by folding it.
- **Projection** — the materialized, fast-to-read state folded from the event log.
- **Idempotency key** — a deterministic id making repeated processing of one logical event a no-op.
- **Transactional outbox** — write to-be-published events to a DB table in the same transaction as the state change, then relay them, to dodge the dual-write problem.
- **Exactly-once (relative to a decision)** — a delivery is processed and published once even across crashes, anchored to a committed inbox decision.
- **Hash chain** — entries linked by `H(prev_hash ‖ entry)` so any edit breaks all later links.

- **Canonical serialization** — a deterministic byte encoding so a hash is reproducible by anyone.
- **LLM-as-judge** — using a stronger model to score another model's output against a rubric.
- **Seam / dependency injection** — a narrow injected interface that makes expensive dependencies fakeable in tests.
- **TDD RED→GREEN** — write a failing test, watch it fail for the right reason, then make it pass.
- **k-sortable ULID** — a 128-bit, time-ordered, globally-unique identifier stored as `uuid`.

Every claim in this document corresponds to code in `src/sentinel/` and a passing test in `tests/`. The system is designed so it earns the right to touch production one graduated grant at a time — and so the guarantees that make that safe are proven, not promised.